STATE, NOT SYNTAX: HARNESS OPTIMIZATION FOR SMALL-MODEL MULTI-TURN TOOL CALLING

A Thesis

Presented to

The Faculty of the Department of Computer Science

California State University, Los Angeles

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

in

Computer Science

By

Sanskar Thapa

December 2026

© 2026

Sanskar Thapa

ALL RIGHTS RESERVED

The thesis of Sanskar Thapa is approved.

Dr. Mohammad Pourhomayoun, Committee Chair

[PLACEHOLDER: Committee member name]

[PLACEHOLDER: Committee member name]

Dr. Chengyu Sun, Department Chair

California State University, Los Angeles

December 2026

ABSTRACT

State, Not Syntax: Harness Optimization for

Small-Model Multi-Turn Tool Calling

By

Sanskar Thapa

Small language models can run on a laptop and call tools for a user, but they often fail when a later call depends on an earlier one. This thesis studies Gemma-4-E2B-it, a model with about two billion effective parameters, on 200 multi-turn tasks from the Berkeley Function Calling Leaderboard. A task is correct only when a simulated backend ends in the right state, such as a car's engine being on.

The best setup leaves the weights fixed. A harness, the program around the model, writes calls as JSON text, records the current state, stops repeated calls, and shows a short card advising the next move. Writing the tool list more compactly on that harness gave the best result: 88 of 200 (44.0%) in its better run and 84 of 200 (42.0%) in the other. The original list scored 71 and 72 of 200 in the same two runs, so the gain held up on these tasks.

The model mainly fails at choosing the next step, not at writing a call, and forcing valid syntax made results worse. We did not test why the compact list helps; one likely reason is that the model has less repeated text to read, leaving more room for the state it needs. Without the compact list, the harness raised accuracy from 30.5% (JSON text) to 38.5% in its first run. That gain is suggestive, not significant after correcting for multiple comparisons. Fine-tuning did not beat the untuned model.

TABLE OF CONTENTS

Abstract iv

List of Tables viii

List of Figures x

Glossary xii

Chapter

1. Introduction 1

2. Background and Related Work 3

    Benchmark 3

    Output Channel 3

    Constrained Decoding 4

    Harness-Side Control and Test-Time Compute 4

    Training for Tool Use 4

3. Setup and Methods 5

    Model and Serving 5

    Harness Interventions 5

    Training 7

    Error Labels 7

    Statistics 7

4. Where Small Models Fail 8

    One Call Is Easy; a Chain Is Not 8

    Most Failures Are About the World 8

    Forcing Valid Syntax Hurts 9

5. What Helps: Harness-Side Control 10

JSON Text over the Native API 10
State Summary 10
Stop on Repeats and the Status Card 10
Full Stack 11
The Best Setup: A Compact Tool List 11
Majority Vote 15
How Sure Are We? 15
6. Fine-Tuning Did Not Help 18
Public Data Taught the Model to Stop 18
A Rebuilt Mix Collapsed 19
7. Discussion 21
State, Not Syntax 21
What We Recommend 21
Limitations 22
8. Future Work and Conclusion 23
Future Work 23
Conclusion 23
References 24
Appendix
A. Intervention Details 29
Calls Written as JSON Text 29
The State Summary Block 29
The Stop Rule 29
The Status Card 29

Combining the Full Stack 30

Sampling Five Calls 30

Two-Pass Constrained Decoding 30

The Tool-Name Automaton 30

B. Failure Labels, Exploratory Tests and Training History 31

Planned Comparisons 31

Training History 32

C. Full Scoreboard and Reproducibility 34

Reproducibility 35

Compact Tool List Checks 36

LIST OF TABLES

Table

1. TABLE 1. COMPACT TOOL LIST AGAINST THE ORIGINAL LIST (SAME MODEL, TASKS, AND OTHER HARNESS PARTS). PAIR 1 COMPARES THE COMPACT LIST WITH A FRESH RUN OF THE ORIGINAL LIST. PAIR 2 RUNS THE COMPACT LIST FIRST. "COMPACT ONLY" COUNTS TASKS THE COMPACT LIST SOLVES AND THE ORIGINAL LIST MISSES. *p* IS AN EXACT TWO-SIDED MCNEMAR TEST, JUDGED AT THE 0.05 LEVEL, OUTSIDE TABLE 2. THE SIGN TEST THAT COUNTS EACH TASK ONCE IS IN THE TEXT. 14
2. TABLE 2. EXACT TWO-SIDED MCNEMAR TESTS ON PER-TASK RESULTS (200 TASKS EACH, THE SAME TASKS). "A ONLY" AND "B ONLY" COUNT TASKS THAT ONE CONDITION SOLVES AND THE OTHER MISSES. *p* IS UNCORRECTED; "CORRECTED" MARKS WHETHER IT STAYS BELOW 0.05 AFTER CORRECTING FOR THESE EIGHT TESTS (HOLM'S METHOD). FULL STACK: JSON TEXT + STATE SUMMARY + STOP ON REPEATS + STATUS CARD; MINIMAL RECIPE: JSON TEXT + STOP ON REPEATS + CARD. EXPLORATORY COMPARISONS OUTSIDE THIS FAMILY ARE IN TABLE 4. 17

3. TABLE 3. PRIMARY ERROR PER FAILED TASK (200 TASKS). NATIVE IS THE NATIVE API. JSON TEXT IS CALLS WRITTEN AS JSON TEXT. STOP = STOP ON REPEATS (GATED); UNGATED = STOP ON FIRST REPEAT; SUMMARY = STATE SUMMARY; MINIMAL = MINIMAL RECIPE; FULL = FULL STACK; VOTE = VOTE OF FIVE; CD = CONSTRAINED DECODING; AUTOM. = TOOL-NAME AUTOMATON; SFT = BEST FINE-TUNE. LABELS AS DEFINED IN CHAPTER 3. 31
4. TABLE 4. EXPLORATORY MCNEMAR TESTS, OUTSIDE THE EIGHT-TEST CORRECTION FAMILY (UNCORRECTED *p*; COLUMNS AS IN TABLE 2). 31
5. TABLE 5. EVERY SCORED CONDITION (GEMMA-4-E2B-IT Q4_K_M). THE ID IS THE INTERNAL LOG ID. INTERVAL: WILSON 95%. ROWS OF 20 ARE SCREENING CHECKS, NOT 200-TASK ACCURACIES. 34
6. TABLE 6. RUNS AND LOGS. 35

LIST OF FIGURES

Figure

1. Fig. 1. The multi-turn tool-calling loop studied in this thesis. Grading uses the final state of the simulated backend; whether the calls parse is not enough. Illustrative, not a scored task. 1
2. Fig. 2. Why tasks fail. Each bar is one condition on the same 200 tasks (* marks the first full-stack run; Constr. decod. = constrained decoding; Name autom. = tool-name automaton; Best SFT = best fine-tune; conditions as in Chapter 3); its height is the number of failed tasks, split by primary error label (Chapter 3). Harness control mostly removes empty turns (grey); wrong final states (blue) remain. Forced-valid output and the best fine-tune shift failures into under-calls (orange) and state mismatches. The compact tool list is omitted. 9
3. Fig. 3. File-system tools from the multi-turn tasks, printed both ways. Both panels contain the same tools. The original list repeats the group's shared description on each tool and indents each schema. The compact list states that description once and prints the schemas without indentation. The excerpt is the first two tools of the group; an ellipsis marks the rest. The labels are the measured median lengths of the full opening prompt, 4,923.5 and 2,888 tokens, a 41% cut. 12

4. Fig. 4. Multi-turn accuracy on 200 BFCL v4 tasks, Gemma-4-E2B-it. Each bar is a separate run, not a stack of the bars below it. Bars: share of tasks fully correct; error bars: Wilson 95% intervals. Dark green: harness control on JSON text (the first full-stack run is outlined and in bold); light green: majority vote, the compute-heavy comparison; grey: output format alone; red: forced-valid output; orange: training. Dotted line: JSON-text baseline (30.5%). Top bar: the full stack with a compact tool list, the better of its two runs (Table 1). 15
5. Fig. 5. 20-task gate (the same 20 tasks for every bar; these are not 200-task accuracies). Error bars: Wilson 95% intervals, in tasks. Green: the untuned model with the status card, which sets the bar (17/20). Orange: fine-tuned models, run with the card except the one marked "no card". Grey: the null test. "Layout": continued training on tool lists laid out like the test; "stops cap" and "look-ups": retrains with the stop share capped or look-up episodes added. Every model trained on the rebuilt mix fails (1–2/20), while the null test stays close to the bar. Self-distillation (training on the model's own checker-approved answers to simulated tasks) and its two rebalanced retrains land in between. 19

GLOSSARY

AST Abstract syntax tree; the single-turn BFCL matcher.
BFCL Berkeley Function Calling Leaderboard.
DPO Direct Preference Optimization.
Full stack JSON text + state summary + stop on repeats + status card.
Greedy decoding Always picking the most likely next token (temperature 0). Repeat runs can still differ.
Harness The program around the model that parses calls, runs tools and records state.
JSON text Calls written as a JSON list in ordinary text, which the harness parses. Logs abbreviate this IFFC.
LoRA Low-rank adapters: small trainable matrices added to frozen weights.
McNemar test A paired test using only tasks where two conditions disagree.
Minimal recipe JSON text + stop on repeats + status card.
Native API Structured `tool_calls` from the server's tool interface. Logs abbreviate this NFC.
Null test A training run that should change nothing if the pipeline is sound.
OODA Observe–orient–decide–act; here only the tag printed on the status card.
pp Percentage points.
Q4_K_M A 4-bit weight-quantization format of llama.cpp.
Self-distillation Training the model on its own answers to simulated tasks that a checker marked correct.
SFT Supervised fine-tuning.
Sign test A paired test counting the tasks on which one condition solved more of its runs than the other; tasks with equal counts are dropped.
Slot One request a server processes concurrently with others.
State summary Harness-written snapshot of facts from executed results.
Status card Four-line card prefixed to the latest user message.
Stop on repeats Ends a turn when the model re-proposes executed calls.
Stop target A training response that is an empty list: no further call.
Tool-name automaton Program that constrains only the tool name.
Turn One user request and everything the model does before handing control back.
Vote of five Majority vote over five sampled calls per step.
Wilson interval A confidence interval for a proportion that stays accurate for small counts.
20-task gate Fixed 20-task screening check (bar 17/20); not a 200-task accuracy.

CHAPTER 1

Introduction

To check a calendar or lock a car, a language model emits a *tool call*: a function name plus arguments. A surrounding program, the *harness*, runs the call against a *backend* (here a simulated vehicle or filesystem) and feeds the result back to the model (Fig. 1). Real tasks chain several dependent calls, which we call *multi-turn* tool calling: look up a value, use it, check the outcome, and know when to stop.

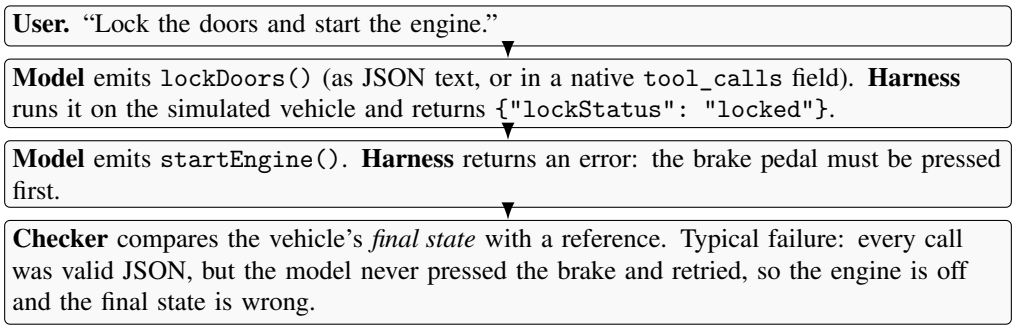


Fig. 1. The multi-turn tool-calling loop studied in this thesis. Grading uses the final state of the simulated backend; whether the calls parse is not enough. Illustrative, not a scored task.

Running this loop locally with a small model keeps private data on the device, works offline and has no per-request cost. The catch is accuracy. We study Gemma-4-E2B-it [1] ("E2B": about two billion effective parameters, meaning the inference footprint; "it": instruction-tuned) on BFCL v4 [2], served with the llama.cpp inference engine [3] on a laptop. Through the server's built-in tool-call field, the native API, it completes 27.0% of 200 multi-turn tasks. The best setup we found, a harness plus a compact tool list, reaches 44.0% (88 of 200) in its better run.

Is the model failing to *write* calls or to *choose* them? Chapter 4 shows it is mostly the latter: tasks fail because a needed tool never ran or the backend ended in the wrong state. We therefore treat the problem as *harness optimization*. The weights stay fixed; we

change the loop around the model: how calls are parsed, what state the harness records and shows back, when it ends a turn, and how the next step is framed. We compare harness control with two other families of fixes: forcing syntactically valid output, and fine-tuning the weights. Our contributions:

1. **A harness plus a compact tool list.** The weights stay fixed. The program around the model records the current state, stops repeated calls, and shows a short card advising the next move. On that harness, writing the same tool list more compactly helps further, and that gain replicated (Chapter 5).

2. **Where the failures are.** The model fails at choosing the next step. A needed call never happens, or the backend ends in the wrong state. The failure analysis is why we built the harness. We did not test why the compact list helps. Forcing each call into valid syntax makes results worse (Chapter 4).

3. **Fine-tuning did not help.** The best supervised fine-tune scores below the untuned model, and preference tuning does not beat it. A fine-tune on the untuned model's own logged answers, which should change nothing if the training steps are sound, stayed close to the untuned model. The rebuilt training set is the leading suspect, and it also differed in training length (Chapter 6).

The rest of this thesis is organized as follows. Chapters 2 and 3 give the background and the setup. Chapter 4 is the failure analysis. Chapter 5 is the harness, including the compact tool list. Chapter 6 is fine-tuning. Chapters 7 and 8 discuss the results and close. The appendices hold the details.

CHAPTER 2

Background and Related Work

This chapter names the benchmark and the ideas the experiments rely on: how a call is written, what it means to force that writing to be valid, what the program around the model can do, and how training a tool-calling model is usually done.

Benchmark

The Berkeley Function Calling Leaderboard (BFCL) evaluates language models as API callers [2, 4]. Its single-turn categories check one emitted call against a reference with an abstract-syntax-tree (AST) matcher. Its multi-turn categories run the model in an agent loop against simulated backends (for example VehicleControl and GorillaFileSystem, a sandboxed Unix-like filesystem) and compare the backends' final state with a reference. A *turn* is one user request and everything the model does before handing control back; each multi-turn task has several turns. We use the *multi-turn base* split (200 tasks, all tools available). Accuracy is the fraction of the 200 tasks whose final state matches.

Output Channel

With the *native API*, the server receives a tool list and the model returns structured `tool_calls`, as in most production APIs. With *JSON text*, the prompt asks the model to write its calls as a JSON list in ordinary text, which the harness parses. Taheri et al. report that small models follow such JSON instructions more reliably than native tool APIs on single-turn BFCL [5]; we use their single-model variant.

Constrained Decoding

Grammar-constrained generation restricts each next token so that output is always valid, for example to a JSON schema or a list of tool names [6, 7]; Yu et al. constrain tool names with a token-level automaton [8]. Format restrictions can also reduce answer quality [9]. Chapter 4 tests whether forcing validity helps in a multi-turn loop.

Harness-Side Control and Test-Time Compute

Context-compression methods summarize long histories [10]; our state summary is instead written by the harness from executed results. Observe–orient–decide–act (OODA) controllers structure agent steps [11]; our status card borrows only the name. Guided templates add a reasoning scaffold before calls [12]. Sampling several answers and voting is a standard way to buy accuracy with compute [13]; our vote of five is that baseline.

Training for Tool Use

Specialist small callers such as xLAM [14] and Hammer [15] are trained on synthetic tool data such as ToolACE [16] and APIGen [17]. *Supervised fine-tuning* (SFT) trains the model to reproduce target responses; we use low-rank adapters (LoRA) [18], small trainable matrices added to frozen weights. *Direct Preference Optimization* (DPO) trains on pairs of preferred and rejected responses [19, 20]. Terms used in this thesis are collected in the glossary before Chapter 1.

The following chapters use only these ideas. They ask where a small model fails, whether a training-free harness with a compact tool list can help, and whether fine-tuning can.

CHAPTER 3

Setup and Methods

Model and Serving

All results use Gemma-4-E2B-it, quantized with Q4_K_M (a 4-bit weight format of llama.cpp), served by `llama-server` on a laptop. Decoding is greedy (temperature 0) except for majority voting. Greedy runs are not identical from one run to the next (Chapter 5). The machine, the server build, and the flags are in Appendix C. Most runs used four parallel server slots, and the native API baseline used eight. This thesis did not test whether those two settings produce the same greedy output.

Harness Interventions

All leave the weights untouched. A task is correct only when the multi-turn checker returns valid. Appendix A gives each rule except the compact tool list. The state summary targets lost state, the stop rule targets repetition, and the status card targets whether to act or stop.

- **JSON text** instead of the native API. The harness parses the calls.
- **State summary.** After each executed call, the harness writes a short `[STATE]` snapshot of facts from real results (current directory, known files, last calls) into the conversation.
- **Stop on repeats.** The harness ends the turn when the model repeats its previous calls, or when it re-proposes calls already executed in this turn after at least six tool steps. An *ungated* variant drops that minimum.
- **Status card.** A four-line card is prefixed to the latest user message. It advises call, ask, finish, or refuse (the card prints finish as `final`) from fixed rules on the

conversation and executed results, and it never sees reference answers. JSON text, stop on repeats, and the card, together, are the *minimal recipe*.

- **Full stack.** JSON text, the state summary, stop on repeats, and the status card, decoded greedily. On its first run this is the harness we compare with JSON text.
- **Compact tool list.** On the full stack, every tool stays in the prompt. The harness states each tool group's shared description once, instead of repeating it, and prints each schema without indentation. It does not remove or retrieve tools. The model, the four pieces above, and the tasks stay the same.
- **Vote of five.** On top of the state summary, at each step the harness samples five candidate calls at temperature 0.7 and executes the most common one. This is the compute-heavy comparison point.
- **Forced-valid decoding.** *Constrained decoding* restricts output in two passes (tool names, then arguments by schema). A *tool-name automaton* constrains only the name, inspired by [8].

A state summary, written from results the harness has already executed, looks like this:

```
[STATE]
cwd:  /workspace/notes
files:  todo.txt, log.csv
last:  cd(folder='notes')->ok
```

A status card, prefixed to the latest user message, looks like this:

```
[OODA]
goal:  lock the doors and start the engine
slots:  lockStatus=locked
last:  startEngine()->error:  brake pedal must be pressed
next:  call
```

Training

We train LoRA adapters on Gemma-4-E2B-it with MLX [21], merge and quantize to Q4_K_M, and evaluate with JSON text. The *rebuilt mix* has 30,000 rows, about 45% newly simulated episodes plus cleaned public data; earlier mixes use ToolACE and sampled xLAM data. One DPO model is trained too. The *gate* is 20 tasks from the same split, run with the status card; the bar is 17/20.

Error Labels

Each failed task gets one primary label: *empty turn* (no call when one was needed), *under-call* (a required tool never ran), or *state mismatch* (tools ran, but the final state differs). A call the harness cannot parse counts as no call.

Statistics

Each reported accuracy comes with a Wilson 95% interval [22], a range that stays reliable when few tasks are correct. When two conditions are compared on the same 200 tasks, we use an exact two-sided McNemar test [23]. It counts only the tasks the two conditions decide differently. The p-value is the chance of a gap at least that large if the conditions were equal.

Eight comparisons were corrected together so that the chance of any false alarm among them stays at 5%. The compact tool list was a separate test, run twice in opposite order. Which comparisons are in the family, and how the two pairs were combined, is in Appendix B.

CHAPTER 4

Where Small Models Fail

One Call Is Easy; a Chain Is Not

On three single-call BFCL categories (840 cases), Gemma-4-E2B-it through the native API scores 772/840 (91.9%, interval 89.9–93.6%; Table 5). *Simple* offers one function and asks for that call (375/400). *Multiple* offers several functions and asks for the right one (183/200). *Irrelevance* offers functions when the query needs no call (214/240). The two parallel-call categories are excluded because earlier runs on this server hit a tool-grammar bug that truncated multi-call output. On multi-turn tasks, the *same model, quantization and server* completes only 54 of 200 (27.0%). The two settings differ in grading as well as length, but the gap is consistent with carrying a chain of calls through a changing state, rather than choosing one call, being the bottleneck.

Most Failures Are About the World

An *under-call* is a task where a required tool never ran. The loop in Fig. 1 is one. The model locks the doors and then tries to start the engine. Both of those calls are valid. The backend says the brake pedal has to be pressed first. The model never presses the brake, so that required call never ran, the engine stays off, and the check fails. A *state mismatch* is the other kind: the tools did run, and the backend's final state still differs from the reference.

Fig. 2 breaks down the failed tasks (full counts in Table 3). For every JSON-text condition, state mismatch is the largest bucket (53–109 tasks). With plain JSON text it is followed by empty turns (41) and under-calls (38); the native API has more empty turns (59) and far fewer under-calls (19). Harness control mainly removes empty turns: the full stack has 20, against 41 for JSON text, while its state mismatches are 68 (60 for JSON

text). Because parse failures can only appear as empty turns, most of what remains is not a formatting problem.

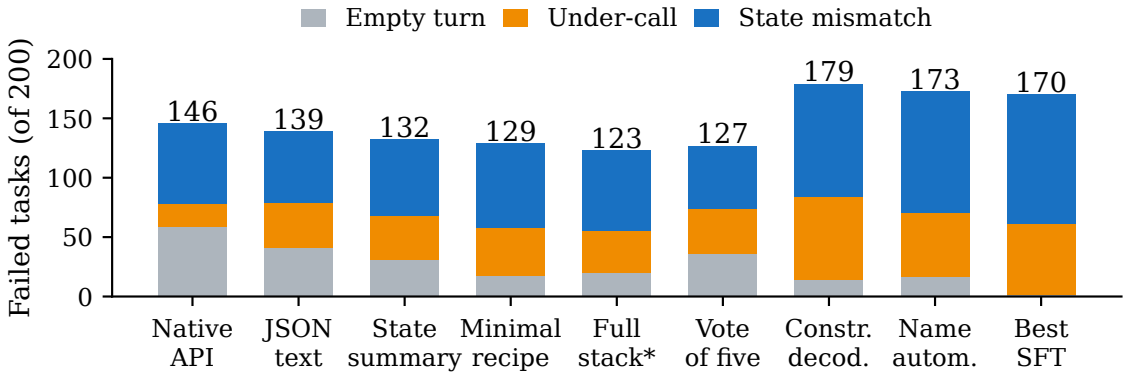


Fig. 2. Why tasks fail. Each bar is one condition on the same 200 tasks (* marks the first full-stack run; Constr. decod. = constrained decoding; Name autom. = tool-name automaton; Best SFT = best fine-tune; conditions as in Chapter 3); its height is the number of failed tasks, split by primary error label (Chapter 3). Harness control mostly removes empty turns (grey); wrong final states (blue) remain. Forced-valid output and the best fine-tune shift failures into under-calls (orange) and state mismatches. The compact tool list is omitted.

Forcing Valid Syntax Hurts

If failures were mostly malformed output, guaranteeing valid output should help. It does the opposite. Two-pass constrained decoding scores 21/200 (10.5%; interval 7.0–15.5%), 20.0 pp below JSON text, with 70 under-calls and 95 state mismatches. The tool-name automaton scores 27/200 (13.5%; 9.4–18.9%), with state mismatch rising to 102. Our two-pass program, which forces each call into valid syntax, approximates rather than reproduces full grammar engines [7], but the effect is large and in the wrong direction: a valid call is not a correct next step.

CHAPTER 5

What Helps: Harness-Side Control

If the bottleneck is choosing the next step, the harness, which sees every executed result, is a natural place to intervene. Every change below is greedy except the vote of five, and none touches the weights. Exact counts and intervals for every condition are in Table 5 (Appendix C).

JSON Text over the Native API

Writing calls as JSON text scores 61/200 against 54/200 on the native API, 3.5 percentage points higher ($p = 0.32$, not significant). Part of the gap may come from serving. A seven-step reasoning template on the native API also scored 61/200, tying JSON text: 19 tasks swap each way, and state mismatches rose to 79. The gap may therefore reflect the shape of the prompt as much as which channel carries the call. We do not claim the channel switch alone as a gain. Later steps are built on JSON text because the harness can parse and inspect those calls directly. The harness levers were not tested on the native API, or combined with the template.

State Summary

The harness-written snapshot scores 68/200, 3.5 percentage points above JSON text ($p = 0.30$). Empty turns fall from 41 to 31 while state mismatches do not (60 to 64), suggesting that the summary mainly keeps the model calling.

Stop on Repeats and the Status Card

The gated stop rule scores 65/200 (+2.0 pp, $p = 0.48$). Stopping at the *first* repeat instead scores 54/200, 11 tasks below the gated rule, and all 37 forced stops fell on tasks that failed. Adding the status card gives the minimal recipe, 71/200 (35.5%, +5.0 pp over JSON text, $p = 0.16$). Relative to stop on repeats alone, empty turns fall from 38 to 17

while state mismatches rise from 60 to 71, so the error labels suggest the card acts mainly as an act-versus-abstain control.

Full Stack

Combining all four greedy changes scores 77/200 (38.5%, interval 32.0–45.4%). Paired against JSON text, it solves 30 tasks that JSON text misses and misses 14 that JSON text solves (Table 2). Against the minimal recipe, the state summary alone, and the vote of five, the differences are within noise. The configuration was assembled after we had seen earlier results on these tasks, so its lead is subject to tuning leakage (Limitations in Chapter 7). The next section keeps this harness and changes only how the tool list is written.

The Best Setup: A Compact Tool List

The best setup is the full stack with the tool list written more compactly. It scored 88 of 200 (44.0%) in its better run and 84 of 200 (42.0%) in the other. This is a repeat on the same 200 tasks, not a test on new tasks. The full stack still sends every tool. The list is long because tools that share a group also share a description, and that description was repeated on every tool, with each schema indented. We state each group's shared description once and print the schemas without indentation (Fig. 3). It does not remove or retrieve tools. Nothing else changes. The model, the four pieces of the full stack, and the tasks stay the same. Like the state summary and the card, this is harness-side control delivered through the context.

Original list. Median opening prompt 4,923.5 tokens.

```
[
  {
    "name": "cat",
    "description": "This tool belongs to the Gorilla file system. It is a simple file system that
allows users to perform basic file operations such as navigating directories, creating files and
directories, reading and writing to files, etc. Tool description: Display the contents of a file of
any extension from currrent directory.",
    "parameters": {
      "type": "object",
      "properties": {
        "file_name": {
          "type": "string",
          "description": "The name of the file from current directory to display. No path is allowed.
"
        }
      },
      "required": [
        "file_name"
      ]
    }
  },
  {
    "name": "cd",
    "description": "This tool belongs to the Gorilla file system. It is a simple file system that
allows users to perform basic file operations such as navigating directories, creating files and
directories, reading and writing to files, etc. Tool description: Change the current working
directory to the specified folder.",
    "parameters": {
      "type": "object",
      "properties": {
        "folder": {
          "type": "string",
          "description": "The folder of the directory to change to. You can only change one folder
level at a time. "
        }
      },
      "required": [
        "folder"
      ]
    }
  }
]
...
```

Compact list. Median opening prompt 2,888 tokens, a 41% cut.

```
This tool belongs to the Gorilla file system. It is a simple file system that allows users to perform
basic file operations such as navigating directories, creating files and directories, reading and
writing to files, etc.
[{"name": "cat", "description": "Display the contents of a file of any extension from currrent
directory.", "parameters": {"type": "object", "properties": {"file_name": {"type": "string",
"description": "The name of the file from current directory to display. No path is allowed. "}},
"required": ["file_name"]}}, {"name": "cd", "description": "Change the current working directory to
the specified folder.", "parameters": {"type": "object", "properties": {"folder": {"type": "string",
"description": "The folder of the directory to change to. You can only change one folder level at a
time. "}}, "required": ["folder"]}}]
...
```

Fig. 3. File-system tools from the multi-turn tasks, printed both ways. Both panels contain the same tools. The original list repeats the group's shared description on each tool and indents each schema. The compact list states that description once and prints the schemas without indentation. The excerpt is the first two tools of the group; an ellipsis marks the rest. The labels are the measured median lengths of the full opening prompt, 4,923.5 and 2,888 tokens, a 41% cut.

We compared that compact list with a fresh run of the original list, then repeated the comparison in the opposite order, with the compact list run first (Table 1). Both pairs use the same model, the same 200 tasks, and the same other harness parts. The 10-task check, and the token and timing counts, are in Appendix C.

In the first pair the compact list scores 84 of 200 (42.0%, interval 35.4–48.9%) and the original list scores 71 of 200 (35.5%, interval 29.2–42.3%). The compact list solves 13 more tasks: 26 that the original list misses, and it misses 13 that the original list solves (58 both, 103 neither). An exact two-sided McNemar test gives $p = 0.053$, just short of the 0.05 level.

The second pair is the confirmation. The compact list scores 88 of 200 (44.0%, interval 37.3–50.9%), the best single run, and the original list scores 72 of 200 (36.0%, interval 29.7–42.9%). The compact list solves 26 tasks that the original list misses and misses 10 that it solves (62 both, 102 neither), $p = 0.011$, which clears the 0.05 level. The gain replicated. These 71 and 72 of 200 are the same runs as the 35.5% and 36.0% full-stack reruns.

The test we rely on counts each task once across both pairs. The compact list does better on 33 tasks and the original list on 17, with 150 ties (exact two-sided sign test, $p = 0.033$). The gain is modest, about 7 points. Repeat runs still move individual tasks: between its two runs, the original list changed result on 15 tasks and the compact list on 18.

TABLE 1. COMPACT TOOL LIST AGAINST THE ORIGINAL LIST (SAME MODEL, TASKS, AND OTHER HARNESS PARTS). PAIR 1 COMPARES THE COMPACT LIST WITH A FRESH RUN OF THE ORIGINAL LIST. PAIR 2 RUNS THE COMPACT LIST FIRST. "COMPACT ONLY" COUNTS TASKS THE COMPACT LIST SOLVES AND THE ORIGINAL LIST MISSES. p IS AN EXACT TWO-SIDED MCNEMAR TEST, JUDGED AT THE 0.05 LEVEL, OUTSIDE TABLE 2. THE SIGN TEST THAT COUNTS EACH TASK ONCE IS IN THE TEXT.

	Compact only	Original only	Both	Neither	p
Pair 1	26	13	58	103	0.053
Pair 2	26	10	62	102	0.011

We did not test why the compact list helps; one likely reason is that the model has less repeated text to read, leaving more room for the state it needs. The compact list is also text in the context. Mean context per request falls by about 30%. Invalid tool names stay at zero for both lists.

Fig. 4 puts this setup on the same scale as the other conditions. The top bar is the compact tool list, the better of its two runs.

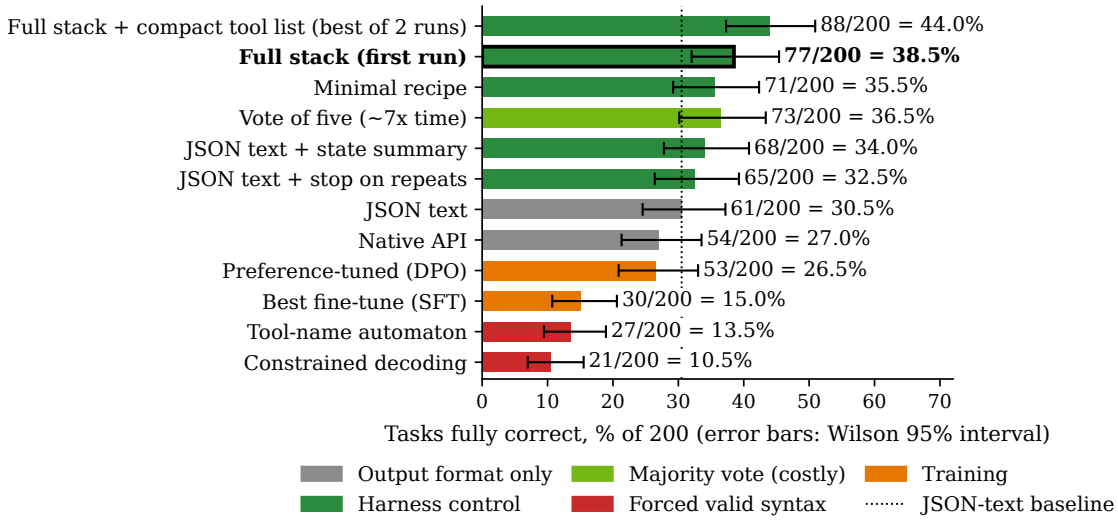


Fig. 4. Multi-turn accuracy on 200 BFCL v4 tasks, Gemma-4-E2B-it. Each bar is a separate run, not a stack of the bars below it. Bars: share of tasks fully correct; error bars: Wilson 95% intervals. Dark green: harness control on JSON text (the first full-stack run is outlined and in bold); light green: majority vote, the compute-heavy comparison; grey: output format alone; red: forced-valid output; orange: training. Dotted line: JSON-text baseline (30.5%). Top bar: the full stack with a compact tool list, the better of its two runs (Table 1).

Majority Vote

The compact list stays greedy. Voting over five samples on top of the state summary scores 73/200 (36.5%), four tasks fewer than the first full-stack run, which solves 24 tasks the vote misses and misses 20 it solves ($p = 0.65$). The vote takes about 7× as long to generate as the greedy full stack, from a single sampling seed. A greedy harness matches it at a fraction of the time, which matters on a local device.

How Sure Are We?

The compact tool list is the one change we ran twice, and that modest gain replicated. We did not label the failures of the compact-list runs, or test the best setup against JSON text directly. The other harness gaps are less firm. Each harness accuracy comes with a range that extends about six to seven percentage points on either side (Table 5). A gap smaller than that can be noise. Of the eight planned comparisons

(Table 2), two harness gaps fall below 0.05 on their own: the full stack against JSON text ($p = 0.023$) and the minimal recipe against the native API (McNemar $p = 0.033$). Neither stays below 0.05 after correcting for the eight tests. The order of the harness rows is therefore tentative, and the full stack's gain of 8.0 percentage points over JSON text is suggestive.

Repeat runs of the full stack do not agree. It scored 77 of 200, then 71 of 200 and 72 of 200. These 71 and 72 of 200 are the same runs as the 35.5% and 36.0% full-stack reruns. A swing of that size is as large as most gaps between harness rows, so a difference from a single run should not be ranked. Against JSON text, the first of those later runs is ahead by 5.0 percentage points (27 tasks only that run solves, 17 only JSON text, $p = 0.174$).

Two contrasts are firm. The best supervised fine-tune solves only 7 tasks that JSON text misses and misses 38 that JSON text solves ($p < 0.001$), and that gap survives the correction. The full stack's interval (32.0–45.4%) does not overlap the best fine-tune (10.7–20.6%) or the forced-valid output. Preference tuning (26.5%) is not separable from JSON text.

TABLE 2. EXACT TWO-SIDED MCNEMAR TESTS ON PER-TASK RESULTS (200 TASKS EACH, THE SAME TASKS). "A ONLY" AND "B ONLY" COUNT TASKS THAT ONE CONDITION SOLVES AND THE OTHER MISSES. p IS UNCORRECTED; "CORRECTED" MARKS WHETHER IT STAYS BELOW 0.05 AFTER CORRECTING FOR THESE EIGHT TESTS (HOLM'S METHOD). FULL STACK: JSON TEXT + STATE SUMMARY + STOP ON REPEATS + STATUS CARD; MINIMAL RECIPE: JSON TEXT + STOP ON REPEATS + CARD. EXPLORATORY COMPARISONS OUTSIDE THIS FAMILY ARE IN TABLE 4.

A	B	A only	B only	p	Corrected
Full stack (38.5%)	Minimal recipe (35.5%)	19	13	0.38	no
Full stack	Vote of five (36.5%)	24	20	0.65	no
Full stack	State summary (34.0%)	24	15	0.20	no
Full stack	JSON text (30.5%)	30	14	0.023	no
Minimal recipe	JSON text	26	16	0.16	no
Minimal recipe	Native API (27.0%)	37	20	0.033	no
JSON text	Native API	22	15	0.32	no
Best SFT (15.0%)	JSON text	7	38	< 0.001	yes

CHAPTER 6

Fine-Tuning Did Not Help

Fine-tuning is the standard way to adapt a small model to a task. Three terms come up in this chapter. The *rebuilt mix* is a later training set of 30,000 rows that combines newly simulated, execution-checked episodes with cleaned public data. The *gate* is a fixed 20-task check drawn from the same split and run with the status card; the bar is 17 of 20, the untuned model's score with the card. The *null test* is a fine-tune that should change nothing if the machinery is sound: one pass over the untuned model's own logged answers.

Here no trained model beat the untuned model. The best supervised fine-tune scored 15.0%, significantly below JSON text, and the DPO model scored 26.5%, within noise of it. The runs are evidence about what goes wrong. The version history is in Appendix B.

Public Data Taught the Model to Stop

The first mixes scored 1/200 (0.5%). A *stop target* is a training response that is an empty list ("no further call"). After a tool result, 78.1% of targets in that data were stop targets, all from ToolACE, so the model learned to stop after its first tool result while BFCL needs further calls. Removing all of those stop targets, and adding recovery examples (an earlier empty reply, then a new request answered with a real call), scored 30/200 (15.0%), our best fine-tune: empty turns nearly vanished (167 to 2), but failures moved to state mismatch (109) and under-call (59).

A Rebuilt Mix Collapsed

The best-validation checkpoint of the rebuilt mix scored 2/20 on the gate (Fig. 5).

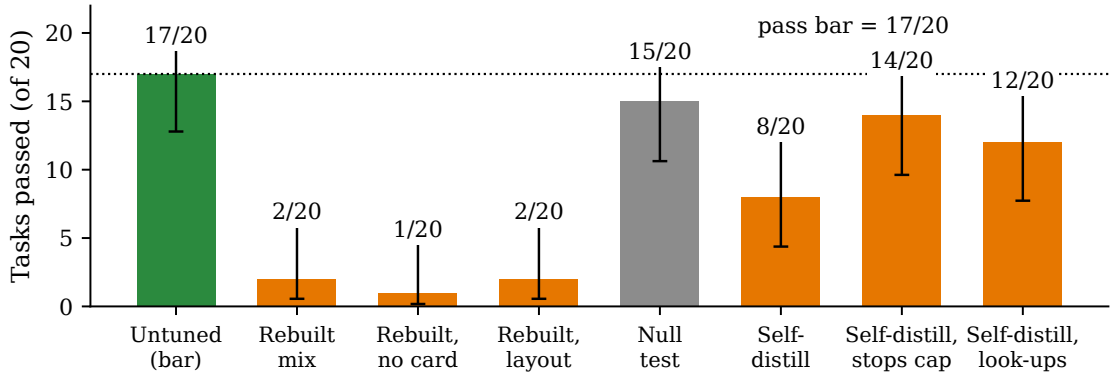


Fig. 5. 20-task gate (the same 20 tasks for every bar; these are not 200-task accuracies). Error bars: Wilson 95% intervals, in tasks. Green: the untuned model with the status card, which sets the bar (17/20). Orange: fine-tuned models, run with the card except the one marked "no card". Grey: the null test. "Layout": continued training on tool lists laid out like the test; "stops cap" and "look-ups": retrains with the stop share capped or look-up episodes added. Every model trained on the rebuilt mix fails (1–2/20), while the null test stays close to the bar. Self-distillation (training on the model's own checker-approved answers to simulated tasks) and its two rebalanced retrains land in between.

Validation loss did not predict the collapse. The collapsed model usually made one call and stopped, even when a setup step was needed (looking up a stock symbol, changing directory, logging in), and invented missing values such as an order number. Without the card it scored 1/20, and continued training on tool lists laid out like the test still scored 2/20. To test the pipeline, the null test trained the untuned model for one pass (39 updates) on 156 of its own logged answers, through the same template, adapter, merge, quantization, and serving steps. It scored 15 of 20, within the noise of the bar of 17 of 20, while the rebuilt mix's 2 of 20 is far outside that bar. The exact ranges are in Appendix B. The null test rules out those pipeline steps as the cause at small update

budgets. The rebuilt mix differs from it in both content and training length (22,000 steps for the evaluated checkpoint), so its data are the leading suspect but not the only one.

Self-distillation trains the model on its own answers to simulated tasks, keeping only the turns a checker marked correct. The suspected cause was a shift in that data, toward stopping and away from look-up calls. Two retrains aimed at that shift did not confirm it. The counts are in Appendix B.

CHAPTER 7

Discussion

State, Not Syntax

Across these conditions, the hard part is choosing the next step from the current state of the world, including whether to stop. Writing a valid call is the easy part. The levers that help are the ones that support that decision: a summary of real results, a gated stop, and a card that mainly controls whether the model acts. We did not test why the compact list helps; one likely reason is less repeated text, leaving more room for the state it needs (Chapter 5). The state summary was meant to put lost facts back in view. What falls, instead, is the number of empty turns. Among the labelled runs, none of the levers reduces state mismatches, and those remain the largest failure type, so the gains come from calling or stopping at the right moment. Tracking the state of the world is still unsolved. Levers that constrain the written form hurt, and training appeared to shift the balance between acting and stopping rather than to improve that tracking.

What We Recommend

On this model and this benchmark, the best setup we found is the full stack with the tool list written more compactly: JSON-text calls, a harness-written state summary, a gated stop rule, a status card, and the same tools with each group's shared description stated once. All of these are greedy and need no training data or training compute. The compact list added about 7 points across two paired runs and cut context per request by about 30%. The minimal recipe, which leaves out the state summary, is a smaller fallback. Before any fine-tune, screen on task-level checks rather than validation loss. Even the best run, the full stack with the compact list, finishes 88 of 200, and a repeat of the same

harness does not pass the same tasks every time. On this benchmark, this model is not yet a reliable multi-step agent.

Limitations

- **One model, one split, one device.** All multi-turn results use Gemma-4-E2B-it on the 200-task base split, on one laptop. The stop threshold, the card's fields, and the training filters were chosen on these same tasks. Other models, and the harder splits of the benchmark, are untested.
- **The harness was assembled on these tasks.** The full stack was put together after earlier results on the same 200 tasks were already known, and the family of eight tests was fixed after that. Its gain over JSON text does not survive that correction.
- **Runs move.** Repeating the same harness does not mark the same tasks correct every time.
- **The compact list.** It ran as two pairs on the same 200 tasks, outside the eight-test correction family. Its 44.0% is the better of two runs, and it has no held-out check.
- **The screening gate was built from the same split.** The 20 gate tasks were picked using earlier per-task results from the same split, and the bar is the score of the setup the gate is screening. It is a screening check, not a confirmation.

CHAPTER 8

Future Work and Conclusion

Future Work

First, *confirmatory runs*: re-score the full stack and the JSON baseline on held-out tasks, the harder BFCL multi-turn splits and a second small model, with matched serving. Second, *retrieval-augmented tool selection*: retrieve the few relevant tools for each step instead of placing every schema in context [4, 24]. The compact list still sends every tool; it is not this selection step. Third, *decision-balanced training data*: vary the look-up and stop shares with everything else fixed.

Conclusion

The best setup is the training-free harness with the same tool list written more compactly. It reached 44.0% (88 of 200) in its better run. The other scored 42.0% (84 of 200). The original list scored 71 and 72 of 200 in the same pairs. These 71 and 72 of 200 are the same runs as the 35.5% and 36.0% full-stack reruns. All of these runs used the same 200 tasks. The gain from that compact list replicated. The model fails at choosing the next step, not at writing a call. The failure analysis is why we built the harness. We did not test why the compact list helps. Without the compact list, the harness raises accuracy from 30.5% to 38.5% in its first run; two reruns scored 35.5% and 36.0%. That gain is suggestive and is not significant after correcting for multiple comparisons. Forcing valid syntax made results worse, and fine-tuning did not beat the untuned model. Among the labelled runs, wrong final states remain the largest failure type. For a local agent, these changes are also far cheaper than voting or training.

REFERENCES

[1] Gemma Team et al. Gemma 4 technical report, 2026. URL https://arxiv.org/abs/2607.02770.

[2] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025.

[3] ggml-org. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2024.

[4] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, volume 37, pages 126544–126565, 2024. doi: 10.52202/079017-4020.

[5] Yalda Taheri, Mohammad Hassan Heydari, Erfan Naaman, and Afsaneh Fatemi. Small reasoning models are instruction followers in function calling. *arXiv preprint arXiv:2608.22472*, 2026.

[6] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[7] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. XGrammar-2: Dynamic and efficient structured generation engine for agentic LLMs. In *Proceedings of the ACM Conference on AI and*

Agentic Systems, CAIS '26, pages 1009–1022, 2026. doi: 10.1145/3786335.3813124.

[8] Minghui Yu, Ke Mu, and Gang Wu. PTC-Decoder: Towards intelligent SLMs on offline resource-constrained edge devices. *arXiv preprint arXiv:2609.30836*, 2026.

[9] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? A study on the impact of format restrictions on large language model performance. In *Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, 2024. doi: 10.18653/v1/2024.emnlp-industry.91.

[10] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. In *International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55321–55375. PMLR, 2026.

[11] Rongfeng Guo, Yinxuan Huang, Yusen Wu, Maoqing Zhong, Yunlu Chen, Meng Tang, Teng Long, and Vincent Tao Hu. From state to action: OODA-Tool for reliable multi-turn tool use. *arXiv preprint arXiv:2608.24368*, 2026.

[12] Hy Dang, Tianyi Liu, Zhuofeng Wu, Jingfeng Yang, Haoming Jiang, Tao Yang, et al. Improving large language models function calling and interpretability via guided-structured templates. In *Empirical Methods in Natural Language Processing*, pages 24426–24442, 2025. doi: 10.18653/v1/2025.emnlp-main.1242.

[13] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

[14] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, et al. xLAM: A family of large action models to empower AI agent systems. In *Nations of the Americas Chapter of the Association for Computational Linguistics*, pages 11583–11597, 2025. doi: 10.18653/v1/2025.naacl-long.578.

[15] Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Xiaoyun Mo, Jiamu Zhou, Cheng Cheng, Yin Zhao, Jun Wang, and Weinan Zhang. Robust function-calling for on-device language model via function masking. In *International Conference on Learning Representations (ICLR)*, pages 85446–85466, 2025.

[16] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, et al. ToolACE: Winning the points of LLM function calling. In *International Conference on Learning Representations (ICLR)*, 2025.

[17] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, et al. APIGen: Automated PIpeline for generating verifiable and diverse function-calling datasets. In *Advances in Neural Information Processing Systems*, volume 37, pages 54463–54482, 2024. doi: 10.52202/079017-1725. Datasets and Benchmarks Track.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large

language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

[19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[20] Sunghee Jung, Donghun Lee, Shinbok Lee, Gaeun Seo, Daniel Lee, Byeongil Ko, Junrae Cho, Kihyun Kim, EungGyun Kim, and Myeongcheol Shin. DiaTool-DPO: Multi-turn direct preference optimization for tool-augmented large language models. In *Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 397–416, 2025. doi: 10.18653/v1/2025.sigdial-1.32.

[21] Apple Machine Learning Research. MLX: An array framework for Apple silicon. https://github.com/ml-explore/mlx, 2023.

[22] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158): 209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[23] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

[24] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large

language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, pages 9695–9717, 2024.

[25] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

APPENDIX A

Intervention Details

Calls Written as JSON Text

Following Appendix B of [5], the tool list is omitted from the API request; the system prompt asks for a JSON list of `{function_name, parameters}` objects in assistant text. Tool results return as user messages. Calls map to the same BFCL function strings as native calling.

The State Summary Block

After each successful tool execution, the harness writes a compact `[STATE]` block from real results (current directory, known files, last executed calls). The last two turns stay verbatim; older raw outputs are dropped. It is not a model-written summary. An example is in Chapter 3.

The Stop Rule

After a tool result: (1) if the proposed call set equals the previous one, stop; (2) else if every proposed call was already executed in this user request *and* at least six non-empty tool steps have occurred in it, stop; (3) otherwise execute. The six-step window resets on a new user request. The ungated variant applies rule (2) with no minimum.

The Status Card

Prefixed to a working copy of the latest user message (the stored transcript is unchanged), stacked on stop on repeats. Fields: `goal` (latest user question, truncated); `slots` (up to eight key–value facts from executed arguments and results); `last` (last executed call and outcome); `next` (advisory: `refuse` if no tools are available, `ask` if the last result signals a missing or invalid argument, `final` if the user's message is a closing

"thanks/done", otherwise `call`). The hint does not constrain decoding and the card never reads reference trajectories. It is not the multi-pass OODA-Tool of [11]. An example is in Chapter 3.

Combining the Full Stack

The state summary, stop on repeats and status card together: the summary is written into the stored transcript after each executed call, and the card is prefixed to a working copy of the latest user message, as above.

Sampling Five Calls

On the state-summary configuration, each step samples five completions at temperature 0.7, majority-votes the `(function_name, arguments)` tuple, executes the winner once, and continues. It is not a learned verifier.

Two-Pass Constrained Decoding

Two requests per call: the first constrained to an empty list or a list of valid tool names, the second constraining arguments with each tool's JSON schema. No native tool grammar is active. This approximates XGrammar-2's tag-triggered dispatch [7] on stock llama-server.

The Tool-Name Automaton

A live finite automaton restricts the tool name to valid names while arguments are free, on JSON text, not stacked on stop on repeats. Execution accuracy here is not comparable to the F1 reported by Yu et al. [8].

APPENDIX B

Failure Labels, Exploratory Tests and Training History

TABLE 3. PRIMARY ERROR PER FAILED TASK (200 TASKS). NATIVE IS THE NATIVE API. JSON TEXT IS CALLS WRITTEN AS JSON TEXT. STOP = STOP ON REPEATS (GATED); UNGATED = STOP ON FIRST REPEAT; SUMMARY = STATE SUMMARY; MINIMAL = MINIMAL RECIPE; FULL = FULL STACK; VOTE = VOTE OF FIVE; CD = CONSTRAINED DECODING; AUTOM. = TOOL-NAME AUTOMATON; SFT = BEST FINE-TUNE. LABELS AS DEFINED IN CHAPTER 3.

Label	Native	JSON text	Stop	Ungated	Summary	Minimal	Full	Vote	CD	Autom.	SFT
Empty turn	59	41	38	40	31	17	20	36	14	17	2
Under-call	19	38	37	31	37	41	35	38	70	54	59
State mismatch	68	60	60	75	64	71	68	53	95	102	109
Failed	146	139	135	146	132	129	123	127	179	173	170

TABLE 4. EXPLORATORY MCNEMAR TESTS, OUTSIDE THE EIGHT-TEST CORRECTION FAMILY (UNCORRECTED p; COLUMNS AS IN TABLE 2).

A	B	A only	B only	p
State summary (34.0%)	JSON text (30.5%)	20	13	0.30
Vote of five (36.5%)	State summary	17	12	0.46
Stop on repeats (32.5%)	JSON text	11	7	0.48
Minimal recipe (35.5%)	Stop on repeats	26	20	0.46

Planned Comparisons

Eight comparisons were planned as one family. The family covers the full stack and the minimal recipe against their baselines, the switch from the native API to JSON text, and the best fine-tune. Four of the eight use the full stack, and the family was fixed after that configuration was built. Four single-lever comparisons were left out of the family (Table 4): the state summary against JSON text, the vote against the state summary, stop on repeats against JSON text, and the minimal recipe against stop on repeats. The

raw p-value is reported, and so is whether Holm's correction [25] changes the conclusion. That correction keeps the chance of any false alarm among the eight tests at 5%.

The compact tool list is not one of those eight tests. It is judged at the 0.05 level. It was run as two pairs, the second in the opposite order. To combine the pairs, count how many of its two runs each list solved for each task, and apply an exact two-sided sign test to the tasks where the counts differ.

Training History

LoRA adapters [18] on Gemma-4-E2B-it with mlx-lm; best-validation checkpoint fused and quantized to Q4_K_M; evaluated with JSON text. The rebuilt mix: 30,000 training / 1,500 validation rows; 45/40/10/5% simulated / APIGen / ToolACE / xLAM; 22% empty-list targets (all after a tool result); 41.2% of rows with multi-call targets; no evaluation tool names copied into simulations. DPO: β=0.2, LoRA, scored on JSON text.

The null test scored 15 of 20 (interval 53–89%), within the noise of the bar of 17 of 20 (64–95%). The rebuilt mix scored 2 of 20 (3–30%), far outside that bar.

Version History

Dropping only mid-dialogue empty targets from the first mix still scored 1/200; deleting trailing empty answers at serving time scored 4/200. After the repaired mix (30/200), two further filtering variants scored 23/200 and 26/200 (Table 5); each changed several filters at once, so they are not single-factor ablations. The rebuilt mix's training was stopped early at step 26,500 of 32,000; the best validation loss (0.030) was at step 22,000, and that checkpoint is the one evaluated. Continued training for 2,500 steps on tool lists laid out like the test scored 2/20.

A Candidate Cause That Targeted Retrains Did Not Confirm

Self-distillation (training on the model's own answers to simulated tasks, keeping only turns the task checker marked correct) scored 8/20; in 6 of 12 failures the model said an ID or price was missing and stopped instead of calling the tool that would supply it. Compared with the null-test data, look-up-type calls (a name-matching heuristic) fell from 37.9% to 19.7% of calls and stop targets rose from 23.1% to 33.2%. Two targeted retrains tested this: capping the stop share scored 14/20, and restoring the look-up share (39.6%) scored 12/20. Neither passed, so the data are *consistent with*, but do not establish, a shift in the decision mix (stop versus continue, look up versus act) as the cause. Leak checks found no evaluation task IDs or tool names in the null-test or self-distillation data.

Self-Distillation Counts

Look-up-type calls: 67/177 in the null-test data versus 78/395 in the self-distillation data; stop targets: 36/156 versus 146/440. Capping the stop share (22.9%) left the look-up share low (20.6%). The look-up-restoring retrain lost 7 tasks the untuned model solves, and empty turns returned (9 of 66 turns). Look-up counts rest on a name-matching heuristic.

Stop Targets in the Rebuilt Mix

Stop targets were 22% of all rebuilt-mix rows, all after a tool result, a different denominator from the first mix's 78.1% (the share of targets *after a tool result* that were stops). The rebuilt-mix models still under-called, so stop share alone does not explain the collapse.

APPENDIX C

Full Scoreboard and Reproducibility

TABLE 5. EVERY SCORED CONDITION (GEMMA-4-E2B-IT Q4_K_M). THE ID IS THE INTERNAL LOG ID. INTERVAL: WILSON 95%. ROWS OF 20 ARE SCREENING CHECKS, NOT 200-TASK ACCURACIES.

ID	Condition	Result	Acc.	95% interval
Multi-turn base, 200 tasks				
84	Compact tool list on the full stack, second pair (best single run)[a]	88/200	44.0%	37.3–50.9
83	Compact tool list on the full stack, first pair[a]	84/200	42.0%	35.4–48.9
78	Full stack, first run	77/200	38.5%	32.0–45.4
11	Vote of five on the state summary	73/200	36.5%	30.1–43.4
85	Full stack, second rerun, original tool list[a]	72/200	36.0%	29.7–42.9
82	Full stack, fresh rerun, original tool list[a]	71/200	35.5%	29.2–42.3
42	Minimal recipe: card on stop on repeats	71/200	35.5%	29.2–42.3
10	State summary on JSON text	68/200	34.0%	27.8–40.8
30	Stop on repeats, gated at six steps (MIN=6)	65/200	32.5%	26.4–39.3
5	JSON text	61/200	30.5%	24.5–37.2
9	Guided template on the native API	61/200	30.5%	24.5–37.2
4	Native API	54/200	27.0%	21.3–33.5
27	Stop on first repeat (ungated)	54/200	27.0%	21.3–33.5
51	DPO, full train	53/200	26.5%	20.9–33.0
15	Best SFT, repaired mix (SFT-v4)	30/200	15.0%	10.7–20.6
46	Tool-name automaton	27/200	13.5%	9.4–18.9
18	SFT, filters on first-turn targets off[†] (SFT-v6)	26/200	13.0%	9.0–18.4
16	Best SFT + state summary (SFT-v4+STATE)	26/200	13.0%	9.0–18.4
17	SFT, extra first-turn and state filters[†] (SFT-v5)	23/200	11.5%	7.8–16.7
8	Constrained decoding, two-pass (CD)	21/200	10.5%	7.0–15.5
14	SFT first mix, trailing empty answers stripped (SFT-v2 strip)	4/200	2.0%	0.8–5.0
12, 13	SFT first mixes, two variants (SFT-v1, SFT-v2)	1/200 each	0.5%	0.1–2.8
20-task gate (same 20 tasks for every row)				
69	Untuned + card: gate bar	17/20		
70	Rebuilt mix, best-validation checkpoint (SFT-v11, step 22,000)	2/20		
71	Rebuilt mix, no card	1/20		
72	Rebuilt mix, layout continue-train (SFT-v11 fmt2500)	2/20		
73	Null test (v13 null)	15/20		
74	Self-distillation, correct turns only (v13-sd)	8/20		
75	Self-distillation, stop share capped (v13-sdb)	14/20		
76	Self-distillation + simulated look-up episodes (v13-lk)	12/20		
Single-call BFCL v4 categories, native API[‡]				
79	Three categories below (weighted)	772/840	91.9%	89.9–93.6
	Simple (one function offered)	375/400	93.75%	
	Multiple (several functions; pick the right one)	183/200	91.50%	
	Irrelevance (no call needed)	214/240	89.17%	

[†]First-turn filters drop or require training examples where the first response is empty or under-specified; state filters drop continuations inconsistent with executed tool results. [‡]The two parallel-call categories are excluded because earlier runs on this server hit a tool-grammar bug that truncated multi-call output. [a]Run 80 (scores.id 59) is the 10-task original list, 5/10. Run 81 (scores.id 60) is the 10-task compact list, 8/10. The recorded calls differed from the first full-stack run on five tasks, so the 200-task comparison used a fresh rerun. Run 82 is the 200-task original list and run 83 the 200-task compact list (pair 1, $p = 0.053$). Runs 84 (compact list, run first) and 85 (original list) are pair 2 ($p = 0.011$). Across both pairs the compact list does better on 33 tasks and worse on 17 (sign test, $p = 0.033$).

Reproducibility

Hardware: MacBook Pro, Apple M4 Pro, 48 GB, with Metal acceleration (Apple's GPU interface). Temperature is 0 except for voting (0.7, N=5, one seed). The native API baseline used eight server slots; the runs below used four (Table 6). BFCL v4 is read through the `bfcl-eval` package (the exact version per run was not logged). Failure labels come from the checker messages in the `eval_errors` table. Fig. 2 and 5 are regenerated from these counts by `paper/figures/src/make_refocus_figs.py`. Fig. 4 uses those same counts; it is a copy of that ladder with the log abbreviations removed from two bar labels. The 91.9% interval is the Wilson formula on 772/840 in the single-call log.

TABLE 6. RUNS AND LOGS.

Run	What	scores.id	Log
78	Full stack, first run	51	`sft/v13/paper_evals/runs/b_fullstack_mtb200.log`
79	Single-call categories		`sft/v13/paper_evals/runs/a_single_turn_gemma.log`
80	10-task original list, 5/10	59	`R81_slim/smoke_off.log`
81	10-task compact list, 8/10	60	`R81_slim/smoke_on.log`
82	200-task original list	61	`R81_slim/full_off.log`
83	200-task compact list	62	`R81_slim/full_on.log`
84	200-task compact list, second pair	63	`R82_confirm/full_on.log`
85	200-task original list, second pair	64	`R82_confirm/full_off.log`

Earlier runs used llama.cpp build 8111 (commit `11c325c6e`). Runs 78 and 80–85 used commit `d344123fe`. The full stack's harness was run at commit `344ce45` and merged as `83f35ee`. Server flags for those runs: `llama-server --jinja -fa on -ngl 99 -c 65536 --temp 0.0 -np 4 -cb -ub 1024`. Opening-prompt lengths are medians from `R81_slim/*.metrics.jsonl`.

Compact Tool List Checks

A 10-task check scored 5 of 10 with the original list and 8 of 10 with the compact list. The recorded calls differed from the first full-stack run on five tasks, so the 200-task comparison used a fresh rerun of the original list rather than that first run (77 of 200).

The context window is 16,384 tokens per request (65,536 shared across four slots). The opening prompt on the original list is about 5,000 tokens. Its median length falls from 4,923.5 tokens to 2,888, a 41% cut (41.3%). Mean context per request falls from 7,107 tokens to 4,990, about 30%. Invalid tool names stay at zero (0 of 1,276 calls with the compact list, 0 of 1,244 with the original list). Invalid arguments are 2 of 1,276 against 9 of 1,244. Format errors are 1 of 1,801 model replies against 0 of 1,773. Mean server time per model request, prompt reading plus generation, with four slots, is 15.4 s with the compact list and 18.8 s with the original list (medians 12.7 s and 15.9 s). Most of the drop is generation (16.2 s to 13.5 s). Prompt reading falls from 2.53 s to 1.86 s.

The second pair shows the same pattern. Mean context per request falls from 7,121 tokens to 5,001, again about 30%, and mean server time per request from 19.4 s to 15.4 s, about 20% faster. Invalid tool names and format errors are zero for both lists. Invalid arguments are 4 of 1,300 calls with the compact list against 5 of 1,255 (0.31% and 0.40%).